Tips for Quiz1 (discussed with Professor Loren yesterday):

- Concisely and directly answer the question. **Paragraphs are bad news**. If you have paragraphs or a super long paragraph, you are probably off base.
- Only provide as much context as needed. **They want to see our understanding of how the question interconnects with other levels of the OS.**
    - E.g. If you were to explain a command in unix, how does it move through the levels of the OS to resolve the command?
    - E.g. Explaining the microkernels significance might require getting into level 1's interrupt architecture.
- All proofs have the following format and should not be more than 4 bullet points/sentences for each section:
    - **Assumptions** to include what we are going to contradict
        - e.g. two processes can be in the critical section at the same time
        - e.g. a process following the ordered resource protocol can hold the highest lock H and acquire another lock
    - **Scenario**, e.g. "Suppose 2 processes are in the Critical section at time t"
    - **Proof** (contradiction)
    - Based on our assumptions, our proof is a **contradiction**
    - **Conclusive statement.**

Exploration 0

Questions

1. Why is kernel correctness important? What is the most common way to assure that the kernel is correct?

   The kernel is one of the most critical components of the operating system, managing hundreds of different processes while running at elevated privilege. It is responsible for several components and tasks including memory management, the CPU, and input/output resources. Due to the high risk of impacts to the system if not secured and configured properly, it is of utmost importance that the kernel is engineered to not only meet security standards but also to be efficient in how it operates.

   Due to the importance of the kernel, it is vital that it is architected in a manner that ensures its correctness and resistance to adversarial action. This is where the principle of levels comes into play. The kernel software is built up as a hierarchy of levels, each one constructed on the ones below. Each layer can also have multiple layers within that follow this same principle. This method gives a systematic way to build the software and ensure it remains correct. Construction of a higher level cannot begin until its foundation is completed.

2. What is the difference between the microkernel and the user kernel? Why make the distinction?
   The microkernel consists of levels 0-4 and includes functions that are executed in privileged (supervisor) mode on a single machine. In addition, it runs directly on the hardware of a single machine. The microkernel provides minimal, essential OS services such as process and memory management, low-level inter-process communications (signaling), and basic scheduling.

   The user kernel consists of levels 5-8 and includes kernel functions that are executed in user mode and distributed over the network, instead of operating directly on a single device. The user kernel provides higher-level OS services such as file system management, network protocols, and user-level services. Due to the level it runs at, it has limited privileges in comparison to the microkernel.

   It is important to make this distinction because only the most trusted components (microkernel) can run in privileged mode. In addition, by moving non-critical services to user mode, a failure in these lower priority services would not directly crash or adversely affect the entire system. Keeping the microkernel simple and minimal aids in its correctness and prevents attacks from adversaries.

3. What is the size (lines of code) of Windows 10, Mac OS 10.1*x*, Linux?

   Windows: 50 million

   MAC OS: 87 million

   Linux: 27.8 million

4. An operating system called QNX claims to have the smallest kernel; look it up and investigate. What did they do to make their kernel so small?

   How QNX Achieves a Small Kernel
   QNX is renowned for its extremely small kernel, especially compared to traditional monolithic operating systems. This is a deliberate result of both architectural and implementation choices designed to maximize modularity, scalability, and efficiency.

   Microkernel Architecture
   Minimal Core Functionality: The QNX kernel (often called `procnto`) includes only the most essential services: CPU scheduling, interprocess communication (IPC), interrupt redirection, and timers. All other services—such as filesystems, device

drivers, and networking—are implemented as user-space processes, not as part of the kernel itself. That makes QNX roughly equivalent to our microkernel.

No In-Kernel Device Drivers: Unlike monolithic kernels, QNX does not include device drivers in the kernel. Drivers run as user-level processes, which keeps the kernel size small and improves system stability (a faulty driver can't crash the kernel).

Modular Design: System architects can include only the components needed for a specific application. For instance, if a filesystem or certain drivers are unnecessary, they can be omitted entirely from the system image, further reducing the kernel and overall OS footprint.

Implementation Techniques:

Efficient Code and Data Structures: Successive QNX microkernels have reduced the code required for each kernel call. Data structures are designed for maximum reuse, folding similar features (like various types of signals) into common representations.

Primarily C Implementation: Contrary to some assumptions, QNX achieves its size and performance goals mostly through refined algorithms and data structures in C, not through extensive use of assembly code.

Message Passing: All communication between system components is done via high-speed message passing, which is tightly integrated with the kernel's scheduler. This allows for efficient, priority-based communication without bloating the kernel with additional IPC mechanisms.

Scalability and Customization

Tailored System Images: QNX allows developers to build custom boot images that include only the kernel and the necessary user-space components. Unused executables and libraries can be omitted, and large files can be moved to external filesystems to minimize the in-memory footprint.

Dynamic Component Management: Components can be added or removed at runtime. This flexibility enables QNX to run on both memory-constrained embedded devices and large multiprocessor systems.

5. **(1.5)** Why is kernel security important for servers running LLMs in AI?
   Kernel security is generally of vital importance within the world of designing and implementing an OS. The kernel embodies rules of isolation between users in order

to enable a society of cooperating processes to work efficiently. With respect to LLM servers hosting AI applications, security is even more paramount.

LLMs run across multiple users and cloud environments which necessitate the proper handling of permissions for different processes such as data source fetching or memory of user-specific tasks being run through the LLM. Specific security concerns for LLM servers include: process isolation to prevent different AI models from interfering with each other, memory protection to safeguard sensitive training data and user inputs, resource management for controlled GPU access among competing models, container security for secure model deployment, and prevention of side-channel attacks that could leak information between different user sessions or models.

Generally speaking, kernel security is required to prevent unauthorized access, data leakage, and interference between specific users and models. The multi-tenant nature of LLM services, where multiple users simultaneously interact with resource-intensive AI models, amplifies these security requirements as any kernel vulnerability could potentially expose sensitive user data or allow malicious actors to compromise the entire AI service infrastructure.

AI systems run on a platform that often consists of Linux boxes networked together. The kernel is present on those systems as well.

6. Each kernel level manages a set of OS objects. The file system is a component of Level 6 (see the OS map). It handles all interactions with files and nothing else. Each file is pointed to by a "capability".
    a. What is the difference between file and buffer?
    Answer: A file is a container for a string of bits, with a name and a size. The file itself is an abstraction of a set of fixed-size records stored in disk memory as a tree structure rooted at an index table.

    A buffer is a temporary storage area in main memory. OPEN gathers copies of all a file's component records into a linear file-image in a main memory buffer... The buffer is needed because direct reads and writes to the memory structure would be intolerably slow.

    Source: Systems Abstractions, page 3, Abstract Machines in OS, page 9
    b. What is a capability? How is it encoded as a bit-pattern?

Answer: A capability is a bit-bundle consisting of (type, access, handle) fields. The type field indicates which type of object is pointed to. The access field is a multi-bit code specifying which subset of the class operations may be performed on that object. The handle field is a unique code for the object that distinguishes it from all others of the same type.

From the abstract machines document: c = (B,A,x) is a capability encapsulating the above. It must be unforgeable, where B is the tag indicating the level, A is the access code, and x is the handle.

Sources: Systems Abstractions, page 2, Abstract Machines in OS, page 6

c. Why do files and buffers have different capabilities?
Answer: Files and buffers have different capabilities because they are different types of objects managed at the same level but serving different purposes. File buffers are a second type of object managed by the Files Manager.

Files are persistent storage objects on disk, while buffers are temporary objects in main memory. The CREATE command creates a file capability that can be used as a parameter for the DELETE and OPEN commands. The OPEN command creates a buffer capability that can be used by the CLOSE, READ, and WRITE commands. Different capabilities ensure that operations appropriate to each object type are properly controlled and authorized.

Sources: Abstract Machines in OS, page 9, Systems Abstractions, page 3

Exploration 1

Questions

Use the in-class command language syntax as the basis for your command-related answers. ***Do not use the shell on your OS****. Those shells may be inconsistent with each other, behave slightly differently on different machines, use slightly different syntax, and contain unfounded or unadvertised assumptions in their implementations.*

1. **(2.5)** Kernel functions are not called by normal procedure calls. The normal procedure call instruction, **CALL X**, means to jump to the procedure whose entry point is **X**, saving return information to be restored when the procedure returns (the **RET** instruction). The kernel call is instruction, **KC X**, is restricted to kernel procedures. Whereas normal procedure call lets the caller attempt to jump to any location, kernel call forces entry at the designed kernel entry point. How does this improve security and reliability?

When working with the kernel, we need to understand that this is one of the most protected subcomponents of the operating system. In addition, calls made here are executed in supervisor mode while normal procedure calls are made at the user level. If an adversary were to gain access to execute commands in supervisor mode, we want to prevent them from accessing/overwriting any location in memory. Forcing kernel calls to enter at a designated location in memory works to prevent damage from adversarial action.

Since system programmers design kernel routines to be reliable, correct, and trustworthy, a common practice in these programs is to begin with validity checks on incoming parameter values. If a parameter fails a test, the program immediately returns an error code without attempting to execute the kernel function.

- If a caller could bypass these validity checks—for instance, by tricking the CPU into starting execution after the checks—it could provide invalid data, potentially causing the kernel function to malfunction.
- By forcing all kernel calls to start at their designated entry points, the OS prevents such bypass attempts. This ensures that all necessary checks are performed before the core logic of a sensitive kernel function is executed, safeguarding the kernel from malicious or erroneous input.

Guarantees Correct Execution Start: The reliability, correctness, and trust claims of kernel programs assume that any kernel routine is called by always starting execution at its designated first instruction or entry point.

- By implementing special mechanisms to ensure that kernel calls always start executing at these designated entry points, the OS ensures a predictable and correct initiation of critical functions. This prevents undefined behavior or crashes that could arise if a kernel routine were to start executing from an arbitrary or incorrect internal point.
  - Also, KC is the only way to get into kernel mode as part of calling kernel function. Attempt to directly (CALL <entry>) fails.

2. **(2.5)** Why is it safe to execute a kernel function in kernel mode, but unsafe to execute a user function in kernel mode?
   It's safe to execute a kernel function in kernel mode due to the stringent design principles, validation, and controlled execution environment applied to kernel code. Conversely, it is unsafe to execute a user function in

kernel mode because user-level code lacks these critical safeguards and granting it unrestricted system access poses severe risks to system stability and security.

-Denning wrote – user code could contain malware.

3. **(2.5)** Why is some of the kernel, but not all, designated as microkernel?

   Not all of the kernel is designated as a microkernel due to architectural design choices aimed at creating a more modular, reliable, and secure operating system.

   The principle of least privilege in OS design means giving each component only the minimum permissions and access rights needed to perform its function. This creates a tension in kernel design between performance and security/reliability.

   A microkernel architecture applies this principle by moving as much functionality as possible out of privileged kernel space and into user space, where components run with restricted privileges. However, some kernel components cannot be moved to user space without severe performance penalties or architectural impossibilities.

   What typically stays in the microkernel:

   - Memory management (page tables, virtual memory)
   - Basic inter-process communication (IPC) mechanisms
   - Low-level interrupt handling
   - Process scheduling primitives
   - Hardware abstraction layer basics

   What gets moved to user space

   - Device drivers
   - File systems
   - Network protocols
   - Higher-level system services

   The reason we can't make everything a microkernel comes down to practical constraints:

   - Performance overhead: Context switches between user and kernel space are expensive. Critical operations like memory management need to happen frequently and quickly.
   - Hardware requirements: Some operations require direct hardware access and privileged CPU instructions that only kernel mode can execute.
   - Bootstrapping dependencies: The microkernel needs enough functionality to start the system and manage the user-space services.

- Atomic operations: Certain operations need to be atomic and uninterruptible, requiring kernel-level control.

In a microkernel architecture:

- Only the most essential and critical functions reside within the microkernel. These typically include:
  - o Basic process management: This involves CPU multiplexing (allowing multiple processes to share the CPU by cycling between them in time slices), process creation, and thread management.
  - o Main memory management: This ensures separation of address spaces and can include handling data movement between RAM and secondary storage (though some functions might be outside the kernel).
  - o Inter-process communication (IPC): This provides messaging services between parallel processes, allowing different components to communicate.
  - o Low-level I/O: Basic device management and interrupt dispatching are also part of the microkernel.
- Less critical kernel functions are moved to user space. This includes components like device drivers (beyond the basic low-level I/O), file systems, and network protocols. These are implemented as separate processes or services that run with reduced privileges.

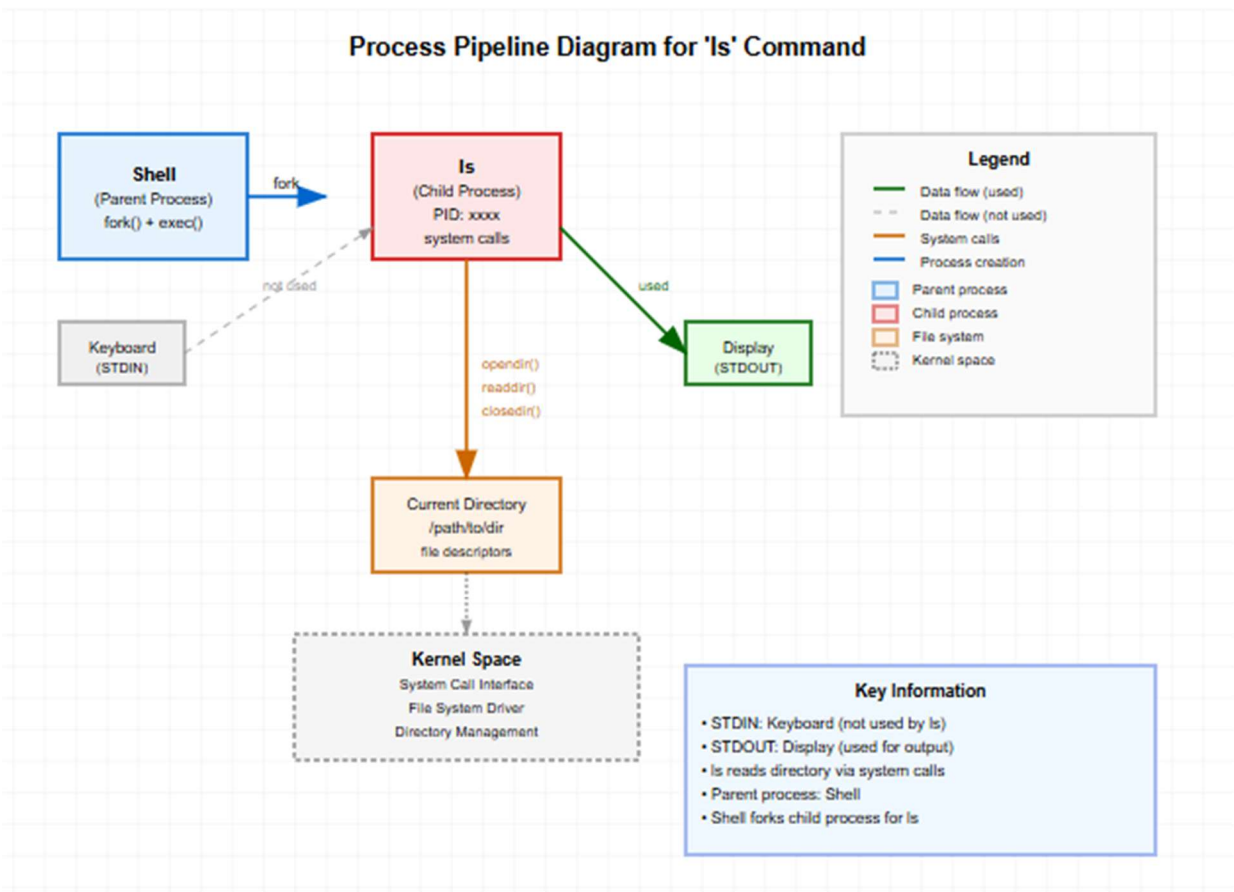
This architectural separation offers several key benefits:

- Improved reliability: If a non-critical service crashes (e.g., a file system or device driver), it does not bring down the entire kernel, as errors in extended functions are "very unusual to crash the kernel".
- Better security: By keeping the microkernel small and in privileged mode, the attack surface is significantly reduced. User processes operate in a non-privileged mode, only switching to kernel mode when explicitly calling a kernel routine.
- Easier maintenance and modularity: Non-critical components can be updated, extended, or replaced independently without requiring modifications to the core kernel. The system is "easier to adapt to different hardware configurations, easier to extend and contract".

The microkernel acts as a core coordinator, handling the fundamental interactions and coordination between these user-space services, which are often communicated using the IPC mechanism.

4. Draw a process pipeline diagram (like those in Chapter 1.2 of Module 1), representing the virtual machines the shell constructs for the typed Unix command **ls**, which displays the contents of your current directory. (In Windows the command is **dir.**) What info objects are provided as standard **IN** and **OUT**? Does **ls** actually use its standard **IN**? Its standard **OUT**? How does **ls** read the current directory? What process is the parent of **ls**?

   A: STDIN: Key board, STDOUT: Display. 'ls' does not read from STDIN. 'ls' reads direcly from system calls such as opendir() or readdir(). 'ls' does use STDOUT. 'ls' reads systems calls, as previously described. It starts with opendir() for file description acquisition where it opens the current directory and returns a file descriptors from the directory. The system call readdir() to return the format of file/directory name, and file type (extra meta data also depends on formatting options in the command line (e.g. -l, -a, etc.). The parent process is the shell. The shell will fork a child process to execute ls.



5. Consider the Unix commands

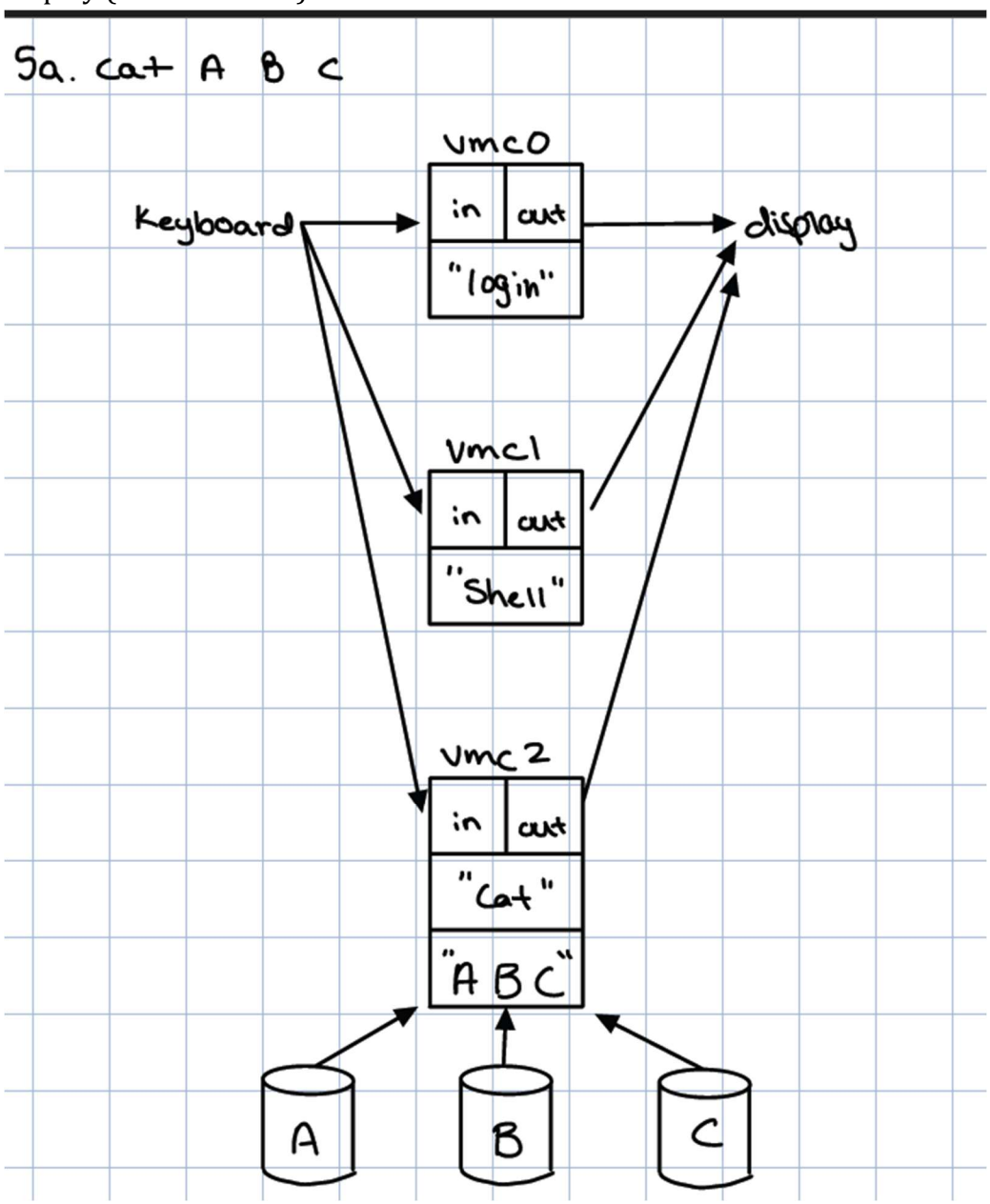
   cat A B C
   cat A
   cat < A

cat < A > D

Look up the definition of **cat** command and, in each of the above cases, explain what is supposed to happen and draw diagrams showing the process structure the OS sets up. What is the difference between the second and third commands?

a. cat A B C

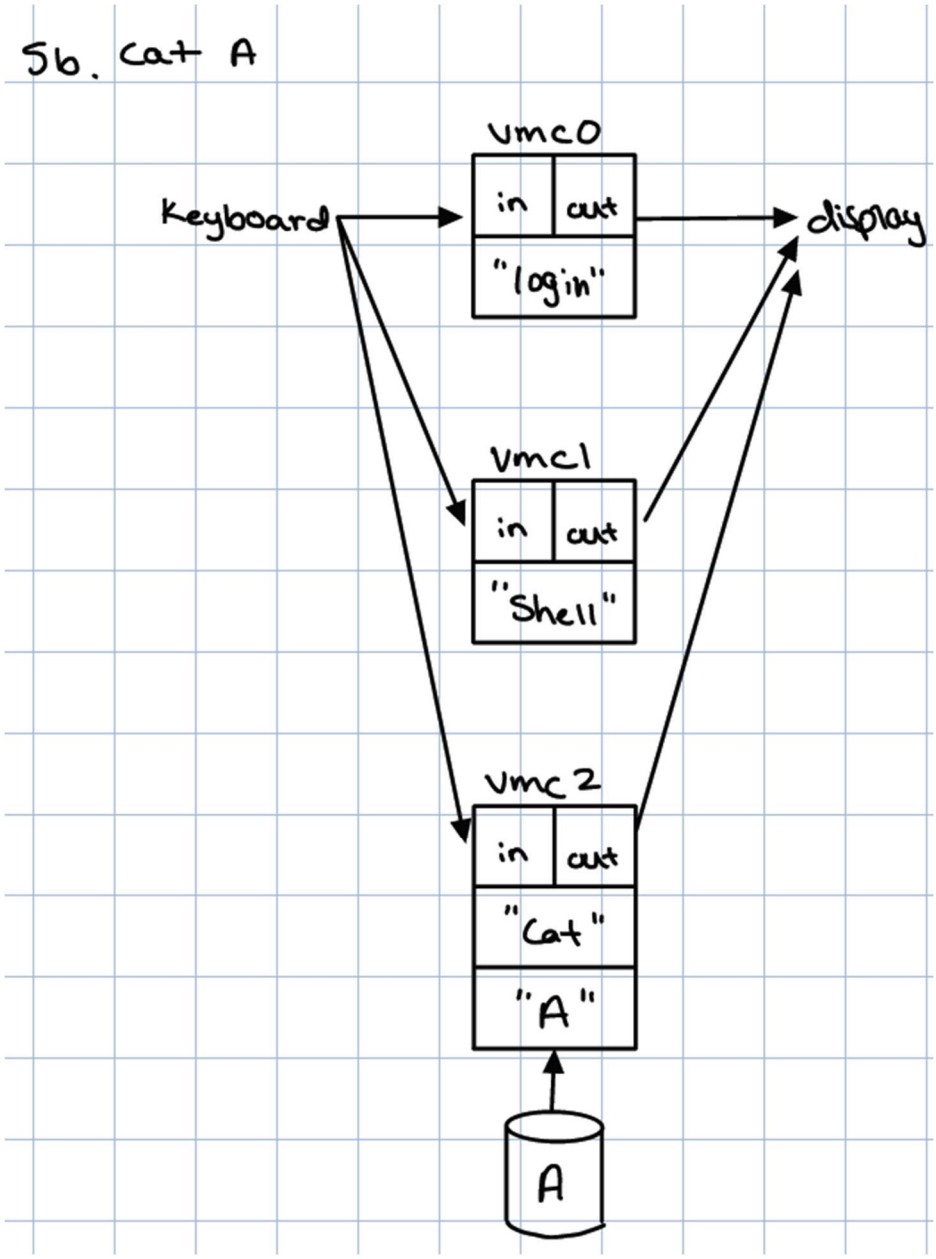
In this case, the shell parses 'cat' as CMD and 'A B C' as ARGS. Cat reads files A, B, and C sequentially and outputs their contents to the display (standard OUT).



b. cat A

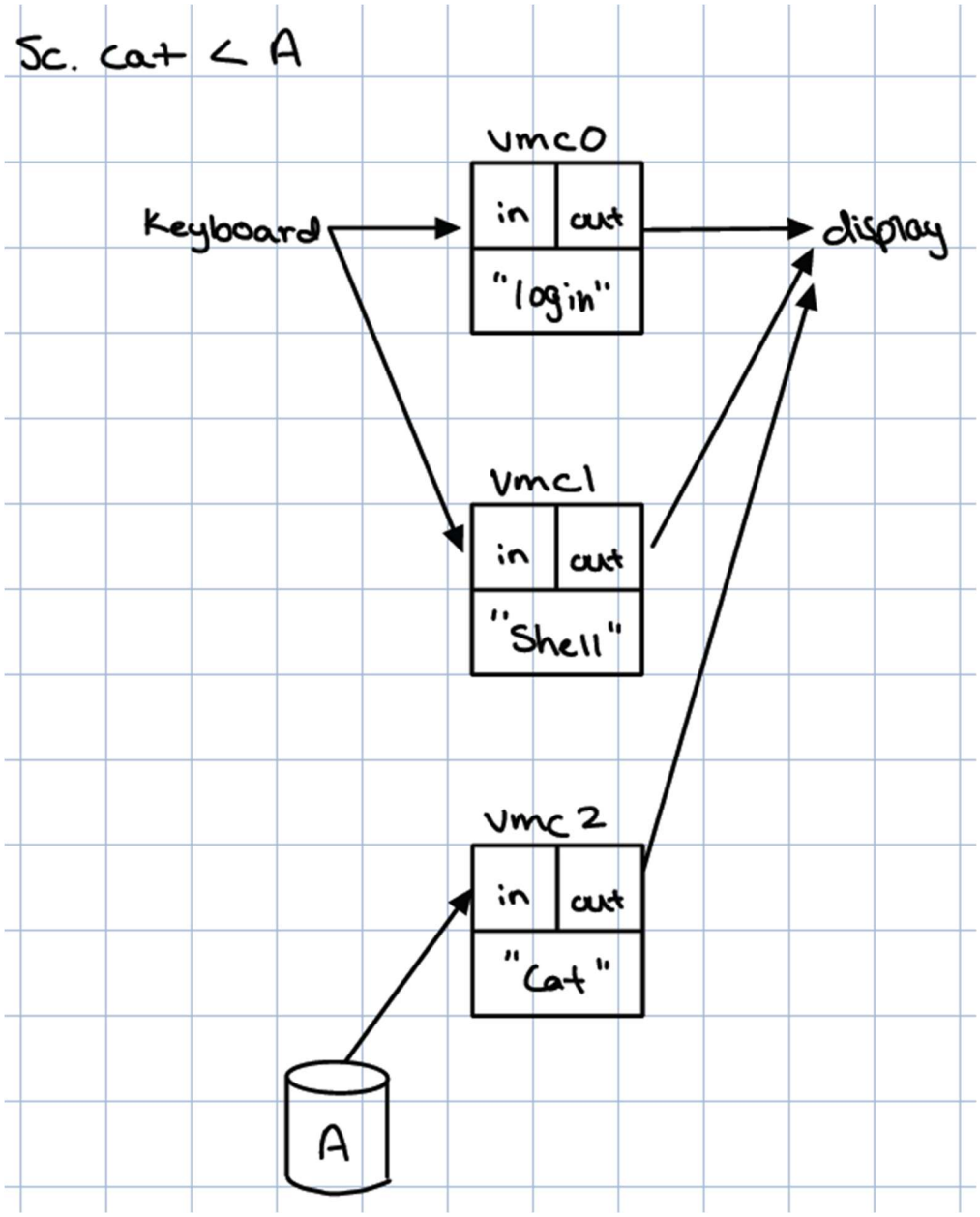
In this case, the shell parses 'cat' as CMD and 'A' as ARGS. Cat reads

file A and outputs its contents to the display.



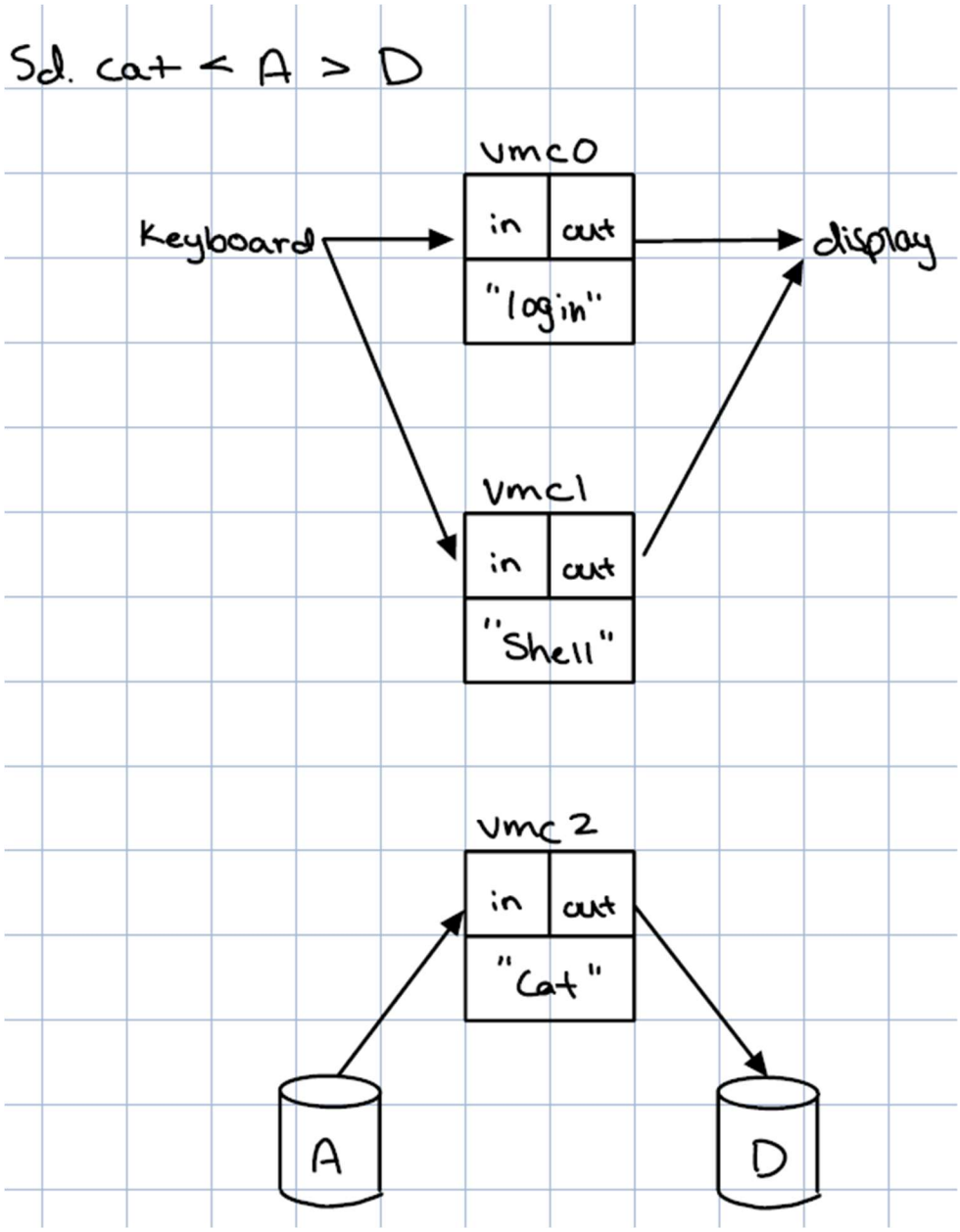
c. cat < A

In this case, the shell parses 'cat' as CMD with A as input redirection. File A becomes standard input for cat, which reads from it and outputs to the display.



d. cat < A > D

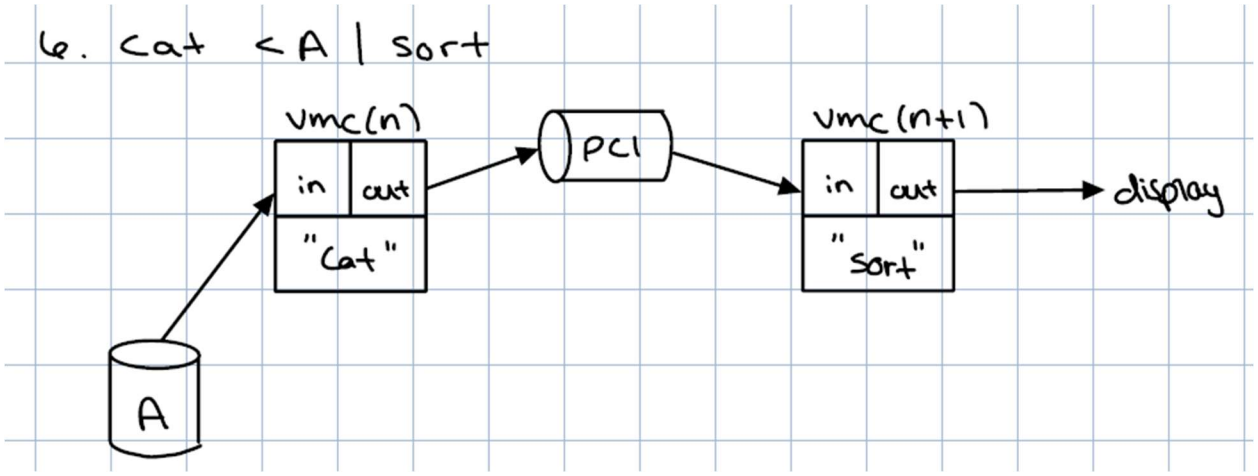
In this case, the shell parses 'cat' as CMD with A as input redirection. File A becomes standard input for cat, which reads from it and outputs to the display.



6. Consider the Unix command

   cat < A | sort

   Draw a diagram showing the process structure the OS sets up. A pipe info object has been added. What is a pipe? What output will be generated by this command line? Where will it go?



The shell reads the string “cat < A | sort” and parses ‘cat’ to be the command to be executed. It then parses the string ‘A’ to be the input redirect, overriding the default input of keyboard. It then parses “|” to pipe the output of the ‘cat’ command to become the input of the ‘sort’ command. The ‘sort’ command will then output to the display (default).

7. (2.0) Where in a command line do file names appear? What happens if the command line specifies a name for a file that does not exist?
The shell (vmc1), which serves as the command-line interface directly interacting with the kernel, parses each command line into four components: name, arguments (args), input, and output. Filenames can be specified as:
2.5
2.5
1. As Arguments (args): Many commands accept filenames directly as arguments.
2. For Input Redirection (<): A filename can be specified as the source for the command's standard input.
3. For Output Redirection (>): A filename can be specified as the destination for the command's standard output.
When a command line specifies a name for a file that doesn't exist, the operation will fail, and the user typically gets an error message. Here's what happens:
How the Directory Manager Handles This (Level 7):
◦ When the shell or a program tries to locate a file by its symbolic name (like text in tbl < text), it makes a SEARCH call to the Directory Manager at Level 7.
◦ The SEARCH operation has a clear rule: it will "Fail if the name does not exist in the given directory" [os99.pdf table 12 (level 7), 265].
◦ Since files must exist within a directory structure to be found, when the

SEARCH operation can't find a matching entry for the filename, it means the file simply doesn't exist or isn't accessible through that path.
What This Means for the Shell and Program:

- When the SEARCH call fails, the shell (or the program making the call directly, like cat opening its argument files) receives an error code back.
- The shell, as the parent process that created the virtual machine for the command, catches this error. Since its job is to "present the response to the user", it displays an error message like "file not found" or something similar.

8. In the command line

   cat < A | sort

   suppose the user inserts "**< B**" immediately after the **cat** command name, intending to read both files **A** and **B** into **cat**:

   cat < A < B | sort

   Is this a valid command? Why or why not? If it is not valid, what command would realize the user's intent?

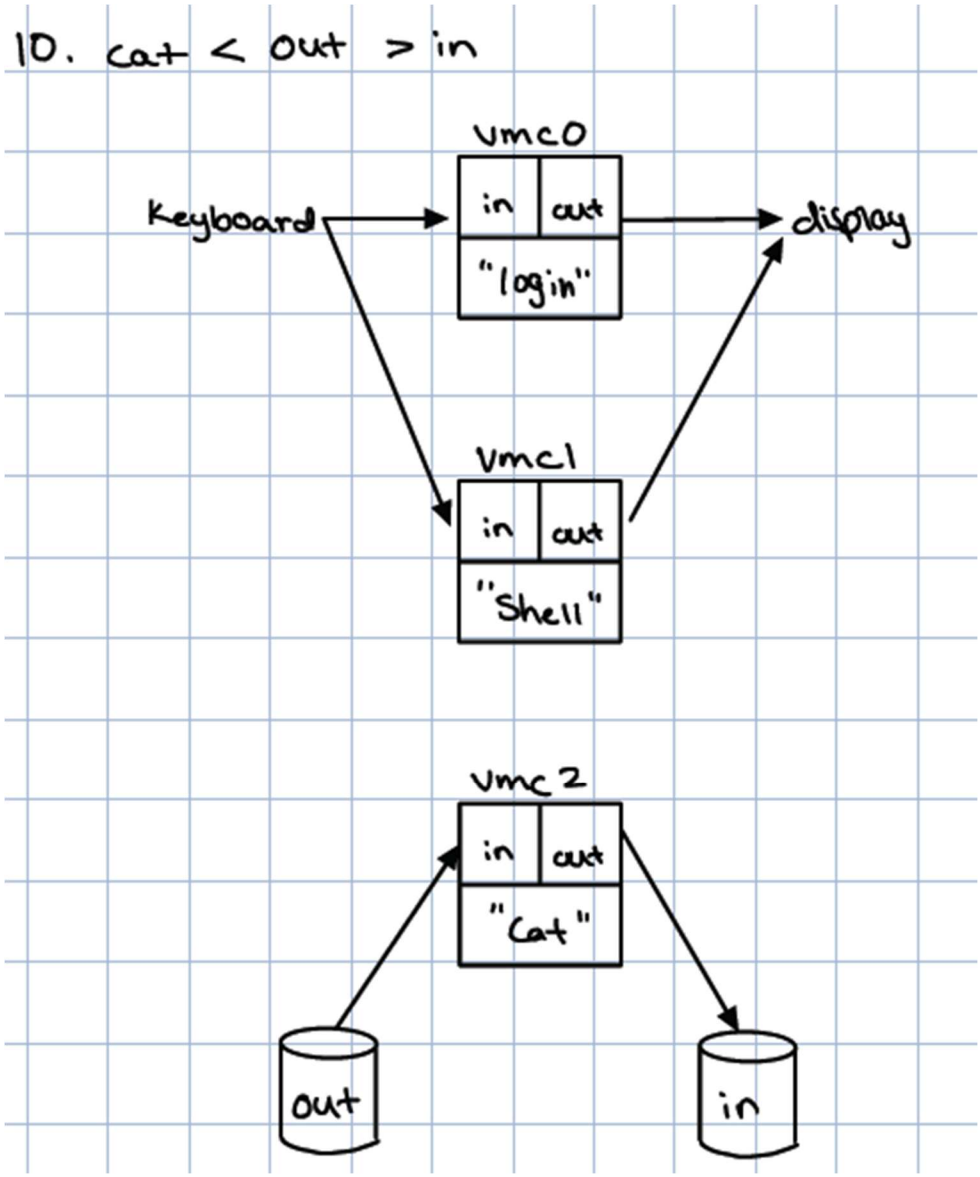
   The command cat < A < B | sort is not valid as shells do not allow multiple input redirections - you cannot have both < A and < B in the same command. The second redirection would conflict with the first. To achieve the intent of reading both files A and B into cat, the correct command would be: "cat A B | sort". This treats A and B as arguments to cat, which will read both files sequentially and pipe the combined output to sort.

9. Where does the shell find the executable codes for the processes named on the command line?

   A: The shell searches the directories listed in the path environment variable. An example being /usr/local/sbin:/usr/local/bin:/usr/bin:/bin. The shell will search the libraries to see if it knows each command in the path (i.e. search for an executable file associated with the command).

10. Does the shell understand the meanings of the commands it executes? For example, would **cat < out > in** be a valid command? Diagram the process structure the OS sets up for this command. If it is valid, what does it do?
    A: The shell does not understand the meaning of "cat" literally. The shell parses the command and creates an environment. "cat < out > in" is syntactically valid. 'cat' is the command, '<' is valid redirect, 'out' would be the stdin, '>' is a valid redirect, and 'in' would be the stdout. Assuming out and in are files, then the shell can find these

files and execute the command. This should take in the contents of 'out' and concatenate it into the contents of 'in.



Exploration 2

Questions

1. The OS provides an **interrupt dispatcher** routine, a short segment of code that looks up the entry point of the selected interrupt in the **interrupt vector**, loads that entry address into the program counter of the CPU, and jumps to the **handler** code. While the dispatcher is working, the interrupts are turned off – that is, the CPU does not check for interrupt signals at the ends of instruction cycles. One of the actions of the handler code is to turn the interrupts back on. Once the handler turns them

back on, the handler is subject to being interrupted by other interrupt signals. Handlers that cannot properly execute if interrupted do not turn them back on until the very end when they exit.

a. **(1.5)** Why must interrupts be turned off while the dispatcher executes?
The OS can't have multiple interrupt signals at a time. The OS needs to handle each exception separately. The selector circuit prioritizes signals, choosing the highest priority one or deferring others. If multiple interrupts were allowed to occur and interrupt the dispatcher, there could be multiple issues like race conditions for interrupt handling or inconsistent/corrupted state of the dispatcher.

Dispatcher sets up activation record for the IH. It cannot be interrupted until this task is done. Otherwise, the AR may not be set up properly and the system is likely to crash.

b. **(2.5)** Why are the instructions that turn interrupts off or on restricted to kernel mode?
If you could turn interrupts off in user mode, the dispatcher could be disrupted and resultant issues like AR setup and beyond could be an issue. Additionally, if the interrupts were turned off, you couldn't handle a page fault.

Instructions that change the state of the system and affect other executing programs are considered sensitive and classified as privileged, therefore instructions to turn off or on interrupts are restricted to the kernel-state. Beyond this, instructions that turn interrupts off create a vulnerability by design such that there is no ability to see another interrupt– and inherently preventing the CPU from being able to respond to vital signals like a page fault, which in turn could lead to system slowdowns and instability.
Also, if interrupts off, you can't make KC (kernel calls).

2. Interrupts have priorities. A signal for a higher priority interrupt can interrupt the handling of a lower priority interrupt.
   a. How are priorities determined?
   Broadly speaking, the priorities of interrupts are "design decisions set by negotiation between software and hardware engineers." The selector handles priorities of signals based on the condition's numbers 0, 1, 2, ..., S-1 with priorities assigned according to the system design (note: priority ordering varies by system architecture). Generally, hardware-detected interrupts for

critical events take priority over software-initiated requests. Hardware events include "divide by zero", "overflow", page faults, etc which are detected by the ALU/MMU and require immediate handling. Lower priority interrupts include software requests like microkernel system calls (KC) which can be deferred.

b. Where in the interrupt hardware are the priorities implemented?
The selector circuit (in the CPU), which is responsible for receiving signals from all sensors and selecting the signal of the highest priority. The Program Status Word (also in the CPU) provides a mask functionality which can hide the lower priority sensor signals until the higher priority signal is handled.

c. **(3.0)** What happens when multiple interrupts arrive in the same instruction cycle?

When multiple interrupts arrive at the same time, the dispatcher will apply a mask to the selector to select the interrupt with the higher priority for immediate handling. The lower priority interrupt is queued and processed once the higher priority interrupt handler has finished being run.

3. A *kernel call* instruction **KC** is used to call a kernel function. The reason for a special instruction is that we want to force entry at the entry point of the kernel code. All the kernel entry points are listed in a kernel vector **KV**. Suppose **KV[72]**="**CREATE_PROCESS**". A user wanting to create a process says "**KC 72**". In general, the instruction "**KC k**" instruction generates an interrupt signal; the KC-handler does a standard procedure call: **CALL KV[k]**.

   a. **(1.5)** Why go to all this trouble? Why not let the user invoke **CALL KV[k]** directly?
   The OS won't allow a user to enter the kernel due to security measures designed by the OS programmers. The kernel completes the request through a hidden process on behalf of the user.

   Allowing the user to invoke a call allowing them to access kernel space directly is a major security concern. An attacker can enter kernel space through a call and then overwrite kernel memory with malware that they have created.
   (How? If I enter a kernel routine, I have to follow its instructions. Those instructions won't include writing malware.)

In order to prevent the user from accessing this protected part of memory, and further masking its exact location, we add a layer of indirection. The only way for the machine to enter kernel mode and subsequently enter the microkernel is with an interrupt created by the selector.
If you do try to CALL KV[k] in user mode, you will be stopped by a memory protection interrupt because KV[k] is in protected kernel memory.

We want to hide the address from the user (bc they can do math on a # and that is bad bc they can do bad stuff). If you try to access the memory even if you had the kernel space memory, you will trigger a memory protection interrupt error. If you don't go via the i-check process where KC calls are triggered via transistors, you can't access that memory location.

4. **(2.5)** If the kernel routine to be called is in the microkernel (such as **CREATE_PROCESS**), the CPU must be put into kernel mode as part of the call, and must be restored to the user mode on return. How does **KC** enter kernel mode? How does the CPU restore the user mode on return?

   When the source code is compiled into assembly, there are certain calls within the code that issue kernel requests. The KC instruction itself generates an interrupt signal, and the interrupt handler is triggered. As part of this interrupt, the CPU hardware automatically enters kernel mode by setting the kernel-state bit to 1 in the PSW. The KC handler ~~creates~~ accesses a kernel vector with an entry point into private kernel space and KC implements the call on behalf of the user.

   When the kernel function completes execution, it returns. This return leads back to the interrupt handler which itself completes with a RET (return) instruction. Since the Activation Record set up by the dispatcher holds the original PSW (which contained the user-mode state), this saved PSW is automatically restored when the activation record is popped from the stack during the return process, automatically switching the CPU back to user mode and allowing the process to continue normal instruction execution.

5. A programmer proposes that the last instruction of a kernel routine prior to return is **ENTER_USER_MODE**.
   a. **(2.0)** Is this valid? Explain. If it is not valid, what is a proper way to return from a kernel call.
      No this is not valid. That is because:
      When a KC k (Kernel Call) instruction is executed, it generates an interrupt signal. As part of the CPU's "interrupt check" at the end of an instruction cycle, if an interrupt is detected, the CPU automatically enters kernel mode by setting the kernel-state bit in its Processor Status Word (PSW) to 1.

During this interrupt handling, the dispatcher software sets up an Activation Record (AR) for the interrupt handler. This AR saves the CPU's current PSW (holding original user state) and the Program Counter (IP) (points to the interrupted instruction).

Security and Control: This mode switching is an orchestrated hardware/OS function, not something that a specific kernel routine itself should explicitly command. Kernel routines are privileged and require protected entry; the exit mechanism is similarly controlled by the system to maintain integrity.

The proper way for a kernel function to return is through a standard RET (return) instruction:

Kernel Function Execution: The kernel function completes its tasks while in kernel mode.

RET Instruction

Activation Record Pop: The RET instruction pops the Activation Record (AR) from the call stack.

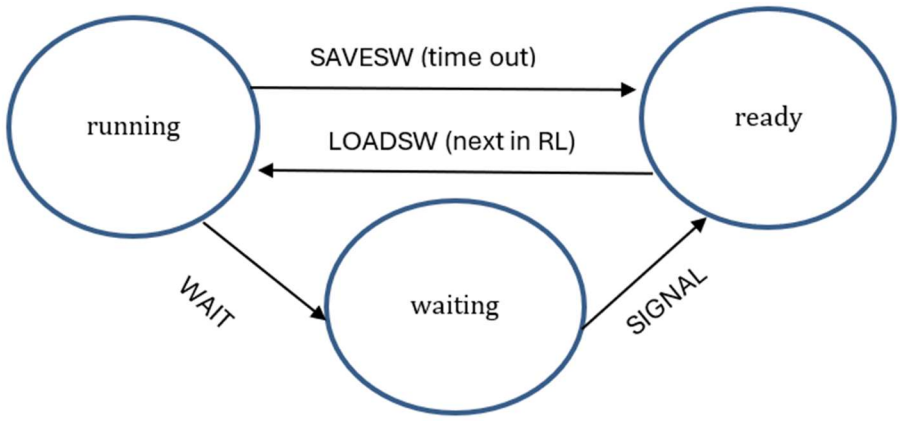
State Restoration and Mode Reversion: Popping the AR restores the saved PSW (which contains the original kernel-state bit value, typically 0 for user mode) and the saved IP to the CPU. This action automatically restores the CPU to user mode (setting the kernel-state bit back to 0) and resumes execution of the interrupted user program at the point it left off. The OS is "coded to switch... back again to 0 when the kernel state is no longer required".

Good. You see that it is redundant in that the proper mode will be restored by the RET instruction. Consider what could go wrong if you entered user mode before reaching the RET instruction. If the interrupted process was in kernel mode it would be an error to return to it in user mode. Moreover in between the time user mode is set and RET is reached, any operation that require kernel mode will be blocked.

Exploration 3

Questions

1. **(3.0)** Draw a diagram showing the three possible process states and allowable transitions. Label transitions with the events that cause them. Describe how these events are caused by particular kernel operations.

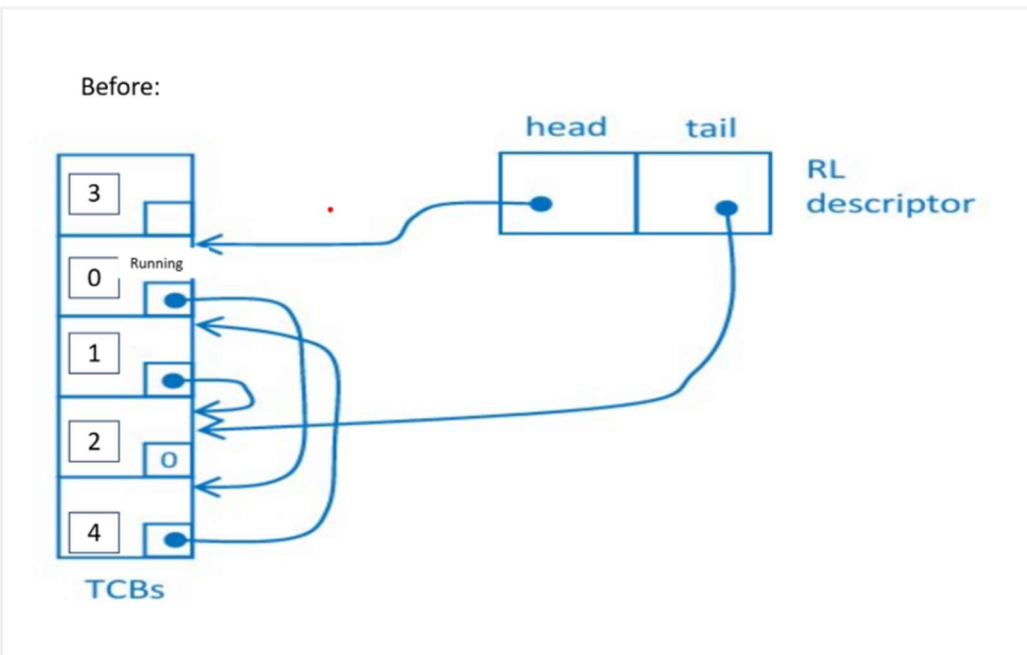


a. Running-SAVESW-Ready: At the end of a time slice, the hardware time triggers a timer interrupt. The timer interrupt handler initiates a context switch, using the "save stateword" (SAVESW) to save the current process's CPU context (state) and process's state into its PCB. The process transitions to the ready state, and its PCB is placed at the end of the ready list. The ready list, via a linked list, chains together the PCBs of all ready processes that are not currently being executed by the CPU.
b. Ready-LOADSW-Running: The OS dispatcher selects the next process from the ready list by triggering the "load stateword" (LOADSW) operation. This restores the process's saved CPU context from its PCB into the CPU. The process moves into the Running state and resumes execution or starts if a new process.
c. Running-WAIT-Waiting: A process moves from the running state to the waiting state when it executes the kernel operation WAIT(s). This operation decrements the semaphore's internal counter. If the new semaphore value is less than zero, the process cannot proceed and is put into the Waiting state. In the Waiting state, the process is paused and cannot proceed until another process signals that the condition is met.
d. Waiting-SIGNAL-Ready: The SIGNAL(s) kernel operation is called by another process and increments the semaphore's counter. The SIGNAL operation indicates that the condition which the waiting process was waiting for has been satisfied. If the semaphore counter remains less than or equal to zero, one process is removed from the semaphore's wait queue and moves to the

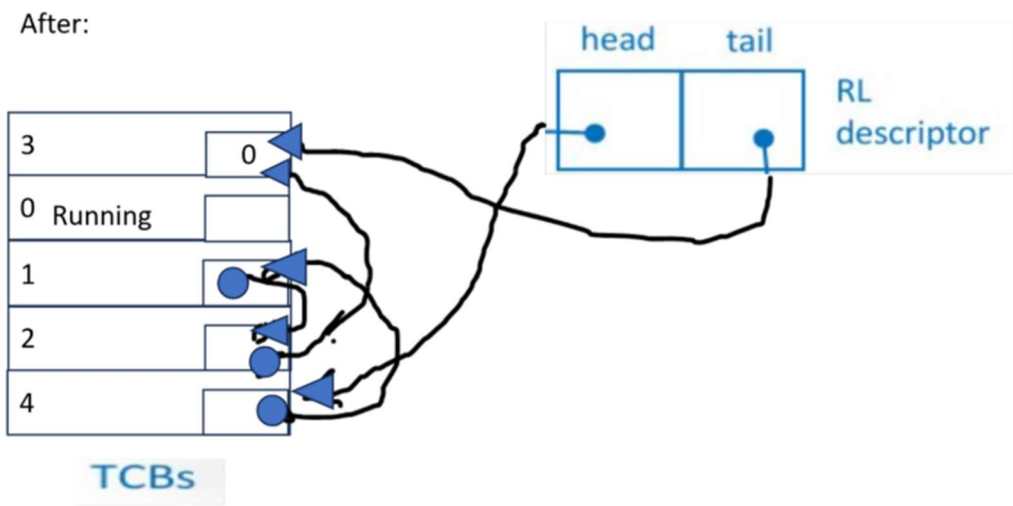
ready state. The kernel updates that process's state to Ready and places its PCB back on the ready list.

2. **(2.5)** The ready list can be implemented as a one-way linked list chaining TCBs (thread control blocks) together. The list itself is represented by a descriptor `RL=(head, tail)`, where `RL.head` is the index of the process at the head and `RL.tail` is the index of the process at the tail. A TCB has a link field (`TLB[i].link`) pointing to the next process after it in the queue. Draw a diagram of a ready list containing three processes waiting while a fourth is running on the CPU. With a before-after diagram, show what changes a context switch makes to this configuration.

   TCBs are normally stored as a linear array. The index in the array is its thread ID. Your diagrams show TCBs in some other order (which is not explained). What does the notation "0 Running" mean?



Before: TCB[0] is executing on the CPU. On the ready list we have TCB [3,1,2,4]. 3 is the head and 4 is the tail. The link list chain is 3,1,2,4.



After: TCB[3] is now running on the CPU. On the ready list we have TCB [0,1,2,4]. TCB 1 is the head and TCB 0 is now the tail. The link list chain is 1,2,4,0.

3. **(1.5)** Suppose that we want 2 or more CPUs to do context switching off a common ready list. Show that a race condition is possible and give an example of a system error that might result. How can this be prevented?

   Since the ready list is a shared resource, that inherently creates potentiality for a race condition if implemented without proper controls. If two processes running on separate CPUs attempt to conduct a context switch simultaneously without synchronization, there exists a potential interlevement of actions that will create unexpected action.

   Multiple CPUs running the same thread: Both CPU 1 and CPU 2 read RL.head simultaneously. They both fetch Process 1 to run, believing it is the next available process. As a result, Process 1 could end up running on two CPUs concurrently. This creates system errors like memory corruption since the same process is writing to memory from multiple locations, inconsistent program state due to concurrent execution of the same instruction stream, and potential lost processes when RL.head updates interfere with each other.

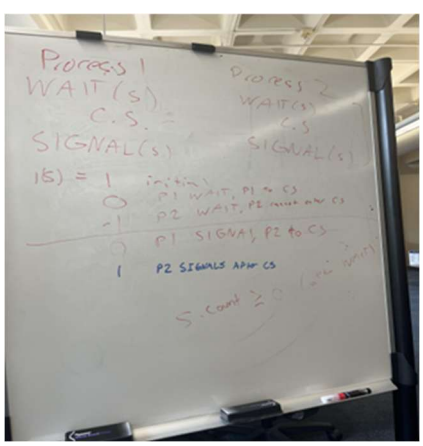
   What about simultaneous manipulation of RL.tail?

To prevent race conditions, we need to build and implement a spinlock. In the Test and Set Lock instruction, you read and return the contents of a memory location without any other interrupts interrupting the instruction cycle. The memory location is locked when it contains a 1, indicating that another CPU is currently accessing the critical section. (What is the critical section?) The memory location is unlocked when it contains a 0, indicating that the resource is available for use. By using the spinlock, only one CPU can enter the critical section at a time, ensuring that the operations on the ready list are atomic and that data corruption and logical errors (like two processes running on one CPU or a lost process) are prevented. The other CPU will busy-wait until the lock is released, then it can attempt to acquire it and proceed.

4. **(3.0)** The mutual exclusion locking protocol uses a semaphore **`s`** with initial counter value **`I(s)=1`**. (Note the important distinction between the semaphore counter **`s.c`** and the initial value of that counter **`I(s)`**). The purpose of the "mutex locking protocol" is to lock a critical section of code, **C**, so that at most one process at a time can execute it. **C** is a block of instructions that manipulate shared data. In each process using the critical section **C**, the protocol is:

```
WAIT(s)
C
SIGNAL(s)
```

Prove that this protocol guarantees mutual exclusion – that is, at most one process can be executing **C** at a time. Hint: assume to the contrary that two processes are in **`C;`** from the definitions of semaphore operations work backward to conclude the only way two processes can be in **C** is a failure of the semaphore**.**



Proof: Assume that Process1 and Process2 are in the critical section, C, at the same time. Given I(s) =1.

Process 1 and Process 2 are in C. Only way a process may enter C after WAIT(s) is if s.c >= 0, so assume s.c = 0 (lowest possible at this point).

Working backwards, Process 1 had to initiate WAIT(s) operation to enter C, so increment s.c. by 1 to equal 1.

Working backwards, Process 2 had to initiate WAIT (s) operation to enter C, so increment s.c. by 1 to equal 2.

This indicates that I(s) = 2, which contradicts our rule that I(s) = 1. Therefore mutual exclusion is guaranteed and the only way two processes can be in C is if the semaphore failed.

5. **(1.5)** A strategy for avoiding deadlock with multiple locks, for example, multiple records in a bank database, is to order the locks -- say ***1, 2, ..., k, ...*** -- and impose the rule that if a process requests a lock while already holding other locks, the number of the new lock must be higher than the numbers of the locks it already holds. Show that this protocol, known as *ordered locking*, cannot deadlock. Hint: suppose that *N* processes deadlock (a circular wait) under this protocol and find a contradiction.

Assume:

1. We have finite N processes that enter k locks
2. Each lock is ordered by increasing number and a process can only acquire a lock of higher number.
   (They are ordered, as above)
3. The N processes deadlock

Proof 1 (Josh and Dylan ordained) (Stephon Approved):

1. Assumption (3) - There is a deadlock → implies all processes are waiting on a lock that another process has acquired ... forever.
2. Assumption (2) - The processes acquire the lock in an order based on descending order → implies each process is waiting on a process that has acquired a lower numbered lock
3. Assumption (1) - There are a finite number processes → implies that eventually, the lowest lock will be acquired by the next process, and so on.
4. The highest lock will eventually get the lock needed from another process.
   → **contradiction** of Assumption (1) - in a deadlock

Not quite right.
Assumption 3: implies that the resource graph has a loop. Assumption 2:

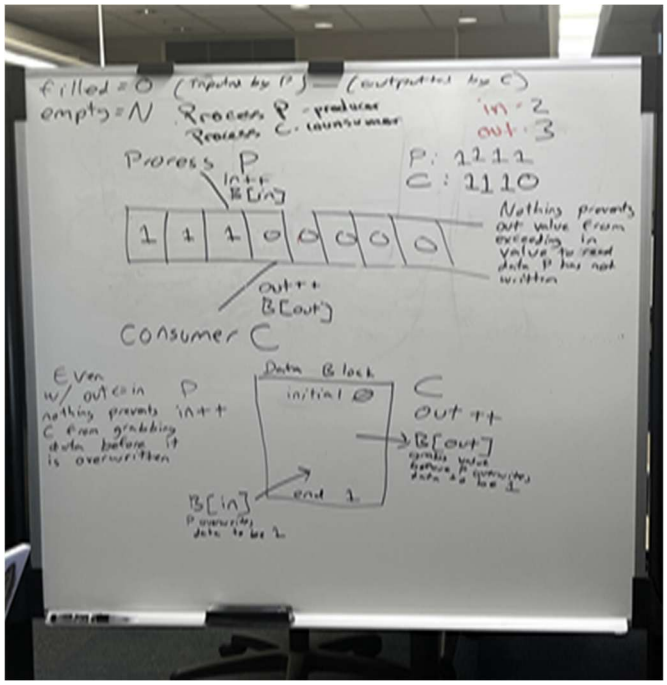
implies that each process is waiting for a higher numbered resource than the one it holds.
That implies that PN (the highest numbered process) must be holding lock LN and asking for L1. --> There is your contradiction.

6. **(1.5)** How can a deadlock occur with the dining philosophers? Would the ordered locking protocol work in this case?

   Consider all philosophers (P) entering and sitting down at the same time. If their immediate response is to grab the fork to their right first, they will all be able to grab one fork. However, when they go to grab the left fork to begin eating, they are unable to as the P to their left has already grabbed that fork ('checked' it out). In this situation where all P's are at the table at the same time, we can enter a deadlock where all P's are waiting on each other to finish eating before they can receive the fork. However, they all will be continuously waiting since they all do not have the required number of forks to begin their process. Important to note that the "program" representing a philosopher says "obtain right, obtain left".
   The ordered lock would work in this scenario so long as we appropriately label each of the locks associated with the forks (1 to N OR 0 to N-1). The philosophers would be required to pick up the lock with the lowest value first, and then the higher valued lock. When they release locks, it would be in reverse order with the higher valued lock being released first. This would effectively break the circular wait condition. You have not shown the break. This rule means that all but one will request forks in order (left, right). The other philosopher will request them in the order (right, left).

7. Let us consider the *producer-consumer* problem. Questions 7, 8, 9, 10 work their way to a practical solution. Here is the problem. One process, ***P***, produces and sends its output bytes to a buffer **B**, and a second process, ***C***, consumes bytes from buffer **B**. We want ***C*** to retrieve all the bytes from the buffer in exactly the same order as ***P*** placed them there, without duplicates or losses. For now, treat the buffer as infinite. There are two pointers, `in` and `out` (both initially 0); ***P*** places its next byte in `B[in]` and increments the pointer, `in++;` ***C*** retrieves its next byte from `B[out]` and increments the pointer, `out++`. Assume the buffer initially contains all 0.
   a. **(3.0)** Show an example in which the byte sequence seen by ***C*** is not the same as the sequence generated by ***P***. This error does not always happen; it is the result of a race condition.
      The C process may get ahead of the P process and attempt to read a block of data from the buffer that has not received input from P yet. The below image demonstrates a situation where the P process is writing "1" to the buffer and

the C process is reading from the buffer. There is nothing preventing the out value from surpassing the in value and having C read a block before P has been written yet. We could end up having P's sequence be 1111 while C's is 1110.



b. **(1.5)** Suppose that the OS could enforce the constraint **`out<=in`** by stopping ***C*** whenever the constraint is violated. Does that eliminate the race?

Not necessarily. For instance, consider the event in where the producer is in the process of writing data to the buffer. The producer writes at location B[in-1] and increments its pointer. At this point, B[in] is pointing to uninitialized space within the buffer. In addition, let us assume that B[in] = B[out] in this scenario. The consumer checks that the above condition is met and begins to read at B[out]. Since the consumer passed the above criteria, it is able to legally read at B[out] even though the producer has not written to that location yet.

This brings up the issue of atomicity. The consumer needs to be able to write to the buffer without the possibility of its action being interrupted by the producer. In the same manner, the producer needs to be able to read legal memory space only after it has been modified by the producer. For this to work, the critical section must be "write B[n]; in++"

Too complicated. If in=out, they can race accessing B[in].

8. Suppose that the OS maintains a counter, **`filled`**, indicating the number of items in the buffer ready for pickup.

a. **(3.0)** When should ***P*** increment the counter `filled`?
Process P should immediately increment the counter filled each time after P has successfully added data to the buffer with B[in].

b. **(3.0)** When should ***C*** decrement the counter `filled`?
Process C Immediately each time after C has successfully grabbed the data from the buffer with B[out].

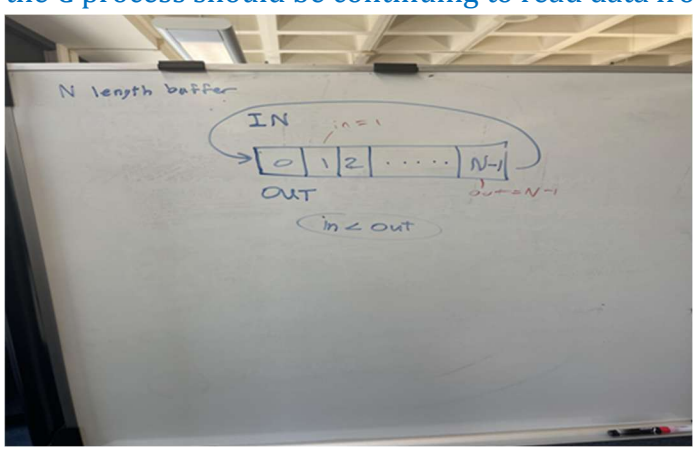
c. **(3.0)** When should the OS stop ***P***?
Assuming an infinite buffer, there are no conditions due to buffer size that would require the OS to put P in the waiting state.

d. **(3.0)** When should the OS stop ***C***?
The OS should stop C once the filled counter is equal to 0, indicating that the buffer does not have any data ready for pickup by C.

9. The previous questions assume the buffer is unbounded. Suppose now it is bounded at length `N`. The `in` and `out` pointers can be recycled, for example `in=(in+1) mod N`.

a. **(3.0)** Why is the constraint `out<=in` now meaningless?

The new recycled pointers now make the values of in and out circular. As they cross the buffer size value N, we will end up in a situation where in<out and the C process should be continuing to read data from the buffer.



Now suppose the OS maintains *two* counters for the bounded buffer: `filled` is the number of items in the buffer ready for pickup (initially `0`), and `empty` is the number of empty slots in the buffer (initially `N`).

b. **(3.0)** When should ***P*** be stopped?
The OS should stop P once the empty counter is equal to 0, indicating that the buffer is full.

c. **(3.0)** When should ***C*** be stopped?
The OS should stop C once the filled counter is equal to 0, indicating that the buffer does not have any data ready for pickup by C.

d. **(2.0)** Why do we need two counters to control the producer-consumer interaction in a finite buffer?

Having only 'filled' and no 'empty' might cause the buffer to overflow, as P puts too much into the buffer.

Having only 'empty' and no 'filled' might cause the buffer to underflow, causing unpredictable consumption.

Right, the constraints are independent. Semaphores test for 0, so you can check each constraint with its own semaphore.

10. **(3.0)** Translate the logic of Question 9 into a semaphore implementation of producer and consumer interacting with the finite buffer. Use two semaphores: **`filled`** (initially **`0`**) and **`empty`** (initially **`N`**).

Semaphore: filled
This will start with a value of 0 and increment every time the producer writes data to the buffer that needs to be read by the consumer. The producer will first wait on the empty semaphore (WAIT(empty)) to block if there are no empty slots remaining. It then conducts its normal operations on the buffer (write item to buffer and increment filled counter). Finally, it will signal to the filled counter (SIGNAL(filled)) that there is an additional item to be read by the consumer.

For example:

```
WAIT(empty);          // block if no slots available
If (entered C):
B[in] = item;         // write item
In = (in + 1) % N;    // increment in pointer
SIGNAL(filled);       // signal that one more item is available
LOOP BACK
```

Semaphore: empty

This will start with a value of N and decrement every time the producer writes something to the buffer (and while those items remain unread by the consumer). The consumer will first wait on the filled semaphore to block if the buffer is empty. It then conducts its normal operations on the buffer (read item from buffer and decrement filled counter). Finally, it will signal to the empty semaphore that an

additional slot in the buffer is available for use (item has already been read and slot is ready to be overwritten and/or deleted).

For example:

```
WAIT(filled);            // block if buffer empty
If (entered C meaning buffer no longer empty):
        Item = B[out];           // read item
        Out = (out + 1) % N;  // increment out pointer
        SIGNAL(empty);        // signal one more empty slot available for writing if
necessary
LOOP BACK
```